

**Data Engineering**

**Azure Databricks**

**Spark**

**vs.**

**SQL Functions**

**Cheat Sheet**



## 1. Syntax Basics

| Aspect          | Spark (DataFrame API)                    | SQL (Spark SQL API)       |
|-----------------|------------------------------------------|---------------------------|
| Language Used   | Python, Scala, Java                      | SQL                       |
| Execution Style | Functional (methods on DataFrame object) | Declarative (SQL queries) |
| Ease of Use     | Ideal for programmatic logic             | Familiarity for SQL users |



## 2. Select and Filter Operations\

| Operation      | Spark Example (PySpark)                              | SQL Equivalent                                        |
|----------------|------------------------------------------------------|-------------------------------------------------------|
| Select Columns | <code>df.select("column1", "column2")</code>         | <code>SELECT column1, column2 FROM table</code>       |
| Filter Rows    | <code>df.filter(col("column") &gt; 10)</code>        | <code>SELECT * FROM table WHERE column &gt; 10</code> |
| Alias Columns  | <code>df.select(col("column").alias("alias"))</code> | <code>SELECT column AS alias FROM table</code>        |



### 3. Aggregations

| Operation              | Spark Example (PySpark)                             | SQL Equivalent                                                    |
|------------------------|-----------------------------------------------------|-------------------------------------------------------------------|
| Group By and Aggregate | <code>df.groupBy("column").agg(avg("value"))</code> | <code>SELECT column, AVG(value) FROM table GROUP BY column</code> |
| Count Rows             | <code>df.count()</code>                             | <code>SELECT COUNT(*) FROM table</code>                           |
| Aggregate Functions    | <code>df.agg(sum("value"), max("value"))</code>     | <code>SELECT SUM(value), MAX(value) FROM table</code>             |



## 4. Joins

| Operation  | Spark Example (PySpark)                    | SQL Equivalent                                                     |
|------------|--------------------------------------------|--------------------------------------------------------------------|
| Inner Join | <code>df1.join(df2, "key", "inner")</code> | <code>SELECT * FROM df1 INNER JOIN df2 ON df1.key = df2.key</code> |
| Left Join  | <code>df1.join(df2, "key", "left")</code>  | <code>SELECT * FROM df1 LEFT JOIN df2 ON df1.key = df2.key</code>  |
| Cross Join | <code>df1.crossJoin(df2)</code>            | <code>SELECT * FROM df1 CROSS JOIN df2</code>                      |



## 5. Sorting

| Operation           | Spark Example (PySpark)                 | SQL Equivalent                                          |
|---------------------|-----------------------------------------|---------------------------------------------------------|
| Order<br>Ascending  | <code>df.orderBy("column")</code>       | <b>SELECT * FROM<br/>table ORDER BY<br/>column ASC</b>  |
| Order<br>Descending | <code>df.orderBy(desc("column"))</code> | <b>SELECT * FROM<br/>table ORDER BY<br/>column DESC</b> |



## 6. String Functions

| Operation       | Spark Example (PySpark)                                    | SQL Equivalent                                               |
|-----------------|------------------------------------------------------------|--------------------------------------------------------------|
| Substring       | <code>df.select(substring("column", 1, 3))</code>          | <code>SELECT SUBSTRING(column, 1, 3) FROM table</code>       |
| String Contains | <code>df.filter(col("column").contains("value"))</code>    | <code>SELECT * FROM table WHERE column LIKE '%value%'</code> |
| String Replace  | <code>df.select(regexp_replace("column", "x", "y"))</code> | <code>SELECT REPLACE(column, 'x', 'y') FROM table</code>     |



## 7. Date and Time Functions

| Operation       | Spark Example (PySpark)                                    | SQL Equivalent                                      |
|-----------------|------------------------------------------------------------|-----------------------------------------------------|
| Current Date    | <code>df.select(current_date())</code>                     | <code>SELECT CURRENT_DATE()</code>                  |
| Add Days        | <code>df.select(date_add(col("date"), 5))</code>           | <code>SELECT DATE_ADD(date, 5) FROM table</code>    |
| Date Difference | <code>df.select(datediff(col("end"), col("start")))</code> | <code>SELECT DATEDIFF(end, start) FROM table</code> |





## 8. Window Functions

| Operation  | Spark Example (PySpark)                                                                               | SQL Equivalent                                                                                                                    |
|------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Row Number | <pre>df.withColumn("row_num",<br/>row_number().over(Window.partitionBy("<br/>column"))))</pre>        | <pre>SELECT<br/>column,<br/>ROW_NUMBER()<br/>OVER<br/>(PARTITION BY<br/>column)<br/>AS<br/>row_num<br/>FROM<br/>table</pre>       |
| Rank       | <pre>df.withColumn("rank",<br/>rank().over(Window.partitionBy("column"<br/>).orderBy("value")))</pre> | <pre>SELECT<br/>column,<br/>RANK()<br/>OVER<br/>(PARTITION BY<br/>column<br/>ORDER BY<br/>value) AS<br/>rank FROM<br/>table</pre> |



## 9. Null Handling

| Operation           | Spark Example (PySpark)                          | SQL Equivalent                                                   |
|---------------------|--------------------------------------------------|------------------------------------------------------------------|
| Drop Null Rows      | <code>df.na.drop()</code>                        | <code>DELETE FROM table<br/>WHERE column IS NULL</code>          |
| Replace Null Values | <code>df.na.fill("value",<br/>["column"])</code> | <code>SELECT<br/>COALESCE(column,<br/>'value') FROM table</code> |



## 10. Miscellaneous

| Operation        | Spark Example (PySpark)                              | SQL Equivalent                                            |
|------------------|------------------------------------------------------|-----------------------------------------------------------|
| Distinct Values  | <code>df.select("column").distinct()</code>          | <code>SELECT DISTINCT column FROM table</code>            |
| Sample Rows      | <code>df.sample(fraction=0.1)</code>                 | <code>SELECT * FROM table TABLESAMPLE (10 PERCENT)</code> |
| Create Temp View | <code>df.createOrReplaceTempView("view_name")</code> | Not applicable in SQL directly                            |

